ASEN 5264 Decision Making under Uncertainty
Exam 2: Online Methods and Reinforcement Learning

Instructions: Clearly indicate your final answers and briefly justify numerical answers with text or mathematical expressions. If you do not understand how to do a problem, skip it and move on so that you have time to attempt all problems. You may consult any static source, but you may NOT communicate with any person except the instructor or TA, and you may not use LLMs such as ChatGPT.

Question 1. (10 pts) Suppose that we are using Q-learning to solve an MDP with $S = \{1, 2\}$ and $A = \{\text{a}, \text{b}\}$. The current Q values are given in the table below:

s	a	$Q(s, a)$
1	a	5
1	b	3
2	a	4
2	b	2

If we take step $(s = 1, a = \text{a}, s' = 2, r = 6)$ and use the Q-learning update rule with $\alpha = 0.1$ and $\gamma = 0.9$, which Q values are changed and what are their new values?

Solution: Because we gained experience for s = 1 and a = a, the value for Q(s = 1, a = a) will be updated. Following the Q-learning algorithm,

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left(r + \gamma \cdot \max_{a'} Q(s', a') - Q(s, a) \right)$$

$$Q(s = 1, a = \text{a}) \leftarrow 5 + 0.1 \left(6 + 0.9 \cdot \max_{a'} Q(s = 2, a') - Q(s = 1, a = \text{a}) \right)$$

$$Q(s = 1, a = \text{a}) \leftarrow 5 + 0.1 \left(6 + 0.9 \cdot Q(s = 2, a = \text{a}) - Q(s = 1, a = \text{a}) \right)$$

$$Q(s = 1, a = \text{a}) \leftarrow 5 + 0.1 \left(6 + 0.9 \cdot 4 - 5 \right)$$

$$Q(s = 1, a = \text{a}) \leftarrow \boxed{5.46}$$

Question 2. (20 pts) Consider a three-armed Bernoulli bandit with win probabilities $\theta = [0.3, 0.7, 0.4]$. After 10 pulls of each arm, the number of wins for each arm is $w = [2, 5, 6]$.

(a) What is the probability that the next pull will result in a win (+1 reward) if an epsilon greedy policy is used with $\epsilon = 0.1$? Briefly justify your answer.

Solution:
The greedy approximation of the success probability $\rho_a = \frac{\text{number of wins } +1}{\text{number of pulls } +1}$ are calculated for each arm.

$$\rho_1 = \frac{2+1}{10+1} = \frac{3}{11}$$
$$\rho_2 = \frac{5+1}{10+1} = \frac{6}{11}$$
$$\rho_3 = \frac{6+1}{10+1} = \frac{7}{11}$$

The ϵ-greedy policy will choose the arm with the highest ρ with a probability $1 - \epsilon$, and randomly otherwise. Thus, the probability of winning on the next pull is equal to the following.

$$\begin{aligned} P_{win} &= (1-\epsilon)\theta_3 + \epsilon\bar{\theta} \\ &= (1-0.1)0.4 + 0.1\frac{0.3+0.7+0.4}{3} \\ &= 0.40\bar{6} \end{aligned}$$

(b) What is the probability that the next pull will result in a win (+1 reward) if a UCB policy is used with $c = 1$? Briefly justify your answer.

Solution:
The Upper Confidence Bound (UCB) equation is given below.

$$\text{UCB}(a) = \rho_a + c\sqrt{\frac{\log N}{N(a)}}$$

Because all arms have been pulled 10 times, the exploration term $c\sqrt{\frac{\log N}{N(a)}}$ is the same for all arms, reducing the policy to the argmax of ρ_a. Thus, the policy will result in pulling arm 3, which has a probability of 0.4 to win.

(c) After a very large number of pulls, which policy will have the highest average payoff? epsilon-greedy or UCB? Briefly justify your answer.

Solution:
In the limit as the number of pulls goes to infinity, the UCB policy converges to the optimal θ^* value, since the exploration term will converge to zero. However, if ϵ remains fixed, the ϵ-greedy policy will not converge to the optimal θ^* value because there will always be an ϵ chance of choosing a suboptimal arm. Thus, the UCB policy will have the highest average payoff.

Question 3. (20 pts) Consider a candidate reinforcement learning algorithm for MDPs with integer states and actions that keeps track of a large table, Q, where each row corresponds to a state and each column to an action. Let $Q[s, a]$ be the entry in row s and column a. The algorithm works as follows:

```
1: Initialize Q to be a matrix of zeros.
2: Initialize s to be the initial state.
3: loop
4:     a ← argmax Q[s, a] (Break ties randomly).
5:     Take action a, receive reward r, and observe the next state s'.
6:     Q[s, a] ← Q[s, a] + 0.1(r − Q[s, a])
7:     s ← s'
8:     if s is a terminal state then
9:         Reset s to the initial state.
10:    end if
11: end loop
```

(a) Should this algorithm be considered model-based or model-free? Why?

Solution: This algorithm is "model free" because it does not learn an MDP model (the reward and transition distributions) to solve, but instead directly estimates a function Q that the policy is based on.

(b) Should this algorithm be considered on-policy or off-policy? Why?

Solution: This is an "on-policy" algorithm because updates to the learned model, Q, use only data from the current policy.

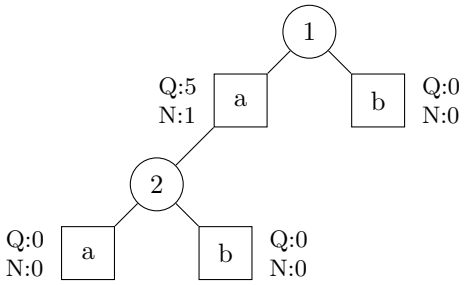
(c) What is a major limitation of this algorithm related to exploration? Briefly justify your answer.

Solution: the policy always takes the action that maximizes Q and only explores randomly if there is a tie. This would make the algorithm susceptible to reaching a local optimum and never exploring other policies.

(d) What is a major limitation of this algorithm related to credit assignment? Briefly justify your answer.

Solution: This algorithm learns a Q function that only takes the immediate reward for a state and action into account - it cannot learn a value function that includes the long-term consequences of an action.

Question 4. (20 pts) Consider the following MCTS tree where state nodes are represented with circles and action nodes with squares. States are represented with integers and there are two actions, a and b. If the generative function is called at any point, use the values in the table to the right and use a discount of $\gamma = 0.9$ if needed.



G(s, a)

s	a	s'	r
1	a	2	6.0
1	b	3	7.0
2	a	4	5.0
2	b	4	6.0

(a) Suppose we run one iteration of MCTS starting from node 1. What is the path of states and actions that the search phase will take including any new state nodes created (e.g. (1, b, 2, b, 5) or (1, a, 4))? Briefly justify why each node is selected.

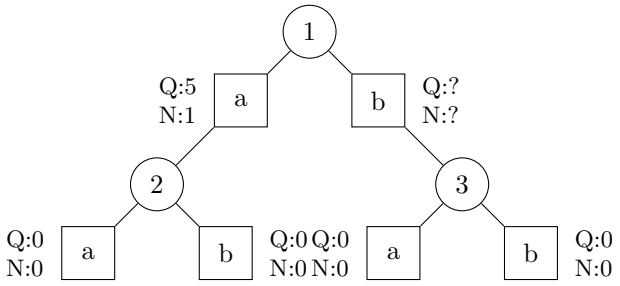
Solution:
MCTS uses the following expression to determine the Upper Confidence Bound (UCB), which is then used to choose actions that maximize the UCB value.

$$\text{UCB}(s, a) = Q(s, a) + c\sqrt{\frac{\log N(s)}{N(s, a)}}$$

Because $N(s = 1, a = \text{b}) = 0$, the UCB value is infinite and the algorithm will choose action b in state 1. From the generative function G, $s' = 3$ so the path would be $(1, \text{b}, 3)$.

(b) Draw any new state and action nodes that would be created in the expansion phase (you can either add to the existing tree or re-draw the tree).

Solution:



(c) Assume the rollout from the new leaf node returns a value estimate of 10. Perform the backup to update the Q and N values at all action nodes nodes on the path (you do not need to copy unchanged values from the existing tree).

Solution:
The updated Q value based on the generative function reward and rollout is given below.

$$\begin{aligned} q(s=1, a=\text{b}) &= r(s=1, a=\text{b}) + \gamma \cdot U(3) \\ &= 7 + 0.9 \cdot 10 \\ &= 16 \end{aligned}$$

Incrementing N value,

$$N(s=1, a=\text{b}) = 1$$

Applying to the MCTS Q update,

$$\begin{aligned} Q(s=1, a=\text{b})_{updated} &= Q(s=1, a=\text{b}) + \frac{q(s=1, a=\text{b}) - Q(s=1, a=\text{b})}{N(s=1, a=\text{b})} \\ &= 0 + \frac{16 - 0}{1} \\ &= 16 \end{aligned}$$